Fatec
Diadema
Luigi Papaiz

Programação para Dispositivos Móveis I

Prof. Marcos Vasconcelos de Oliveira

Marcos.vasconcelos4@fatec.sp.gov.br

Precisão e Formatação de Dados

O controle de exibição é essencial. Utilize String.format para garantir a precisão visual de dados numéricos (Double) na saída do console.

```
fun main() {
    val nota: Double = 8.999999
    println(String.format("%.2f", nota))
}
```

Entrada Bruta

Formata para 2 casas decimais

```
9.00
```

```
1 fun main() {
2     val nota: Double = 8.999999
3
4     println(String.format("%.2f", nota))
5 }
6
```

```
fun main() {
    val nota: Double = 8.999999
    println(String.format("%.2f", nota))
}
```

Fatec
Diadema
Luigi Papaiz

Aula 5 – Funções na Linguagem Kotlin

Prof. Marcos Vasconcelos de Oliveira

Marcos.vasconcelos4@fatec.sp.gov.br

Funções Top Level: A Lógica Libertada

No Kotlin, as funções são cidadãos de primeira classe e não precisam estar presas dentro de classes. Se uma função não retorna valor (Unit), a declaração do tipo de retorno é opcional.

```
fun main() {
    imprimir("Fatec Ferraz")
}

fun imprimir(texto: String) {
    println(texto)
}
```

Definida fora de qualquer classe

Retorno Unit implícito

Note

Comparação: Não existem funções Top Level no Java.

Funções Top Level

- Funções fora de classe
- Não existem funções Top Level no Java
- Função sem retorno (Unit) no Kotlin não é obrigatório declarar.

Funções Top Level

```
fun main() {
    imprimir("Fatec Ferraz")
}

fun imprimir(texto: String){
    println(texto)
}
```

```
fun main() {
    imprimir("Fatec Ferraz")
}

fun imprimir(texto: String): Unit{
    println(texto)
}
```

Unit → Função não retorna valor.

Fatec
Diadema
Luigi Papaiz

Funções Top Level – Código Fonte

- fun main() {
- imprimir("Fatec Diadema")
- }

- fun imprimir(texto: String){
- println(texto)
- }

1. fun main() {
2. imprimir("Fatec Diadema")
3. }

4. fun imprimir(texto: String): Unit{
5. println(texto)
6. }

Fatec Diadema Luigi Papaiz

Funções Top Level

```
fun main() {
    println("${minimo(10,15)} ")
}

fun minimo(a: Int, b: Int): Int {
    return if (a < b) a else b
}
```

Funções Top Level – Código Fonte

```
1. fun main() {
2.     println("${minimo(10,15)} ")
3. }

4. fun minimo(a: Int, b: Int): Int {
5.     return if (a < b) a else b
6. }
```

Fatec
Diadema
Luigi Papaiz

Função de Única Expressão

Quando uma função retorna o resultado de uma única expressão, omita as chaves e use a igualdade.

Padrão (Verborrágico)

```
fun minimo(a: Int, b: Int): Int {
    return if (a < b) a else b
}
```

Single Expression (Conciso)

```
fun minimo(a: Int, b: Int): Int =
    if (a < b) a else b
```

Substitui o bloco de corpo

Funções Top Level

```
fun main() {
    println("${minimo(10, 15)} ")
}

fun minimo(a: Int, b: Int): Int = if (a < b) a else b
```

Funções Top Level – Código Fonte

```
1. fun main() {
2.     println("${minimo(10, 15)} ")
3. }

4. fun minimo(a: Int, b: Int): Int = if (a < b) a else b
```

Fatec
Diadema
Luigi Papaiz

Argumentos Nomeados: Clareza sobre Ordem

Não é necessário decorar a ordem dos parâmetros. Ao nomear os argumentos na chamada, o código torna-se auto-explicativo.

```
fun minimo(a: Int, b: Int): Int ...
```

Chamada Posicional

```
minimo(10, 15)
```

Chamada Nomeada

```
minimo(b = 15, a = 10)
```

Argumentos Nomeados

```
fun main() {
    println("${minimo(10, 15)} ")
    println("${minimo(b = 15, a= 10)}")
}

fun minimo(a: Int, b: Int): Int = if (a < b) a else b
```

Fatec
Diadema
Luigi Papaiz

Argumentos Nomeados – Código Fonte

```
1. fun main() {
2.     println("${minimo(10, 15)} ")
3.     println("${minimo(b = 15, a= 10)}")
4. }

5. fun minimo(a: Int, b: Int): Int = if (a < b) a else b
```

Fatec
Diadema
Luigi Papaiz

Parâmetros Padrão: O Fim do Overloading

Defina valores padrão na declaração. Uma única assinatura substitui múltiplos métodos sobrecarregados.

```
fun potencia(base: Int = 2, expoente: Int = 1)
```

`potencia(2, 3)`	`potencia(10)`	`potencia(base = 2)`	`potencia(expoente = 3)`
	usa expoente = 1	usa expoente = 1	usa base = 2

Parâmetros Padrões

```
fun main() {
   println(potencia(2,3));
   println(potencia(10));
   println(potencia(base=2));
   println(potencia(expoente = 3));
}

fun potencia(base: Int = 2, expoente: Int = 1): Int
{
   return Math.pow(base.toDouble(), expoente.toDouble() ).toInt()
}
```

Parâmetros Padrões – Código Fonte

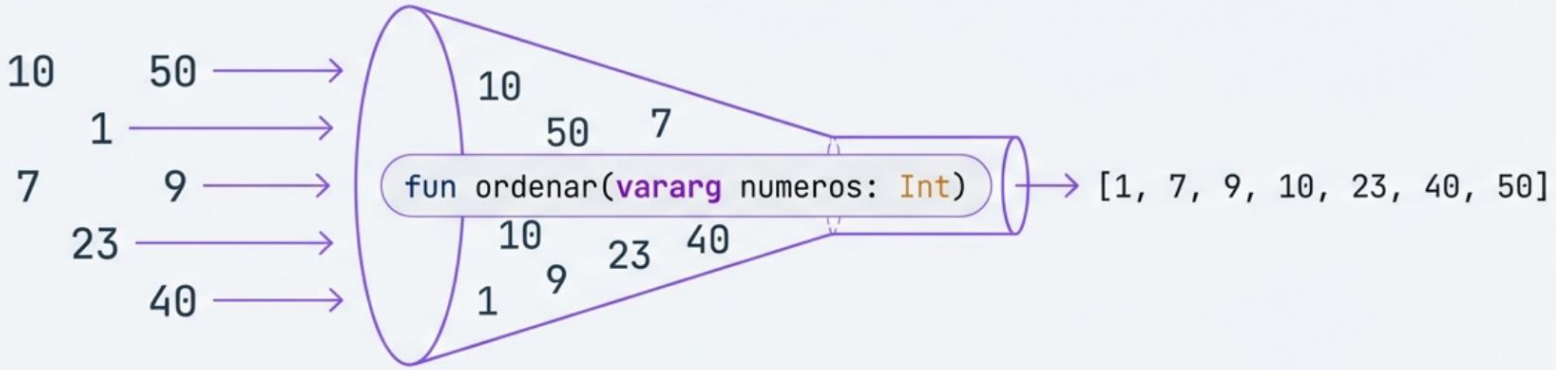
```
fun main() {
    println(potencia(2,3));
    println(potencia(10));
    println(potencia(base=2));
    println(potencia(expoente = 3));
}

fun potencia(base: Int = 2, expoente: Int = 1): Int
{
    return Math.pow(base.toDouble(), expoente.toDouble() ).toInt()
}
```

Fatec
Diadema
Luigi Papaiz

Argumentos Variáveis (Varargs)

O modificador vararg aceita uma quantidade indefinida de entradas e as converte em um Array.




```
fun ordenar(vararg numeros: Int): IntArray {
    return numeros.sortedArray()
}
```

Argumentos Variáveis

```
fun main() {
    for (n in ordenar(10, 50, 1, 7, 9, 23, 40)) {
        println("$n ")
    }
}

fun ordenar(vararg numeros: Int): IntArray {
    return numeros.sortedArray()
}
```

Argumentos Variáveis – Código Fonte

```
1. fun main() {
2.     for (n in ordenar(10, 50, 1, 7, 9, 23, 40)) {
3.         println("$n ")
4.     }
5. }

6. fun ordenar(vararg numeros: Int): IntArray {
7.     return numeros.sortedArray()
8. }
```

Fatec
Diadema
Luigi Papaiz

Funções como Parâmetros

O operador : : cria uma referência à função, permitindo passá-la como dado.

```
fun somar(a: Int, b: Int): Int {
    return a + b
}
```

```
calcular(5, 7, ::somar)
```

O Contrato da Função

```
fun calcular(
    x: Int,
    y: Int,
funcao: (Int, Int) -> Int): Int {
    return funcao(x, y)
}
```

Passando Função como Parâmetro (::)

```
fun main() {
   println(calcular(5, 7, ::somar))
}

fun somar(a: Int, b: Int): Int {
   return a + b
}

fun calcular(a: Int, b: Int, funcao: (Int, Int) -> Int): Int {
   return funcao(a, b)
}
```

Passando Função como Parâmetro (::) – Código Fonte

```
fun main() {
    println(calcular(5, 7, ::somar))
}

fun somar(a: Int, b: Int): Int {
    return a + b
}

fun calcular(a: Int, b: Int, funcao: (Int, Int) -> Int): Int {
    return funcao(a, b)
}
```

Fatec Diadema Luigi Papaiz

O Dilema da Conversão de Dados

Segurança vs. Brevidade. Qual abordagem escolher?

Robustez

```
xxx.toString().toDouble()
```

Compatibilidade máxima.
Tratamento seguro de tipos mistos.

?

Risco

```
xxx!!.toDouble()
```

Código mais limpo, mas com risco de crash se nulo.

A Rota Segura (Indirect)

Ideal para tipos de origem desconhecidos.

```
val number = 42
val doubleValue = number.toString().toDouble()
```

Universaliza o dado

A Rota Direta (Null Safety)

Use apenas com certeza absoluta.

```
val number: Int? = 42
val doubleValue = number!!.toDouble()
```

O operador !! força o compilador a ignorar a verificação de nulo.

Se number == null → CRASH (NullPointerException).

- Em kotlin usar xxx.tostring().toDouble() ou utilizar xxx.!!.todouble()?

1. xxx.toString().toDouble()

- **Descrição**: Primeiro, você chama toString() no valor xxx, convertendo-o para uma representação de string. Depois, você chama toDouble() na string para convertê-la em um Double.
- **Uso típico:** Esta abordagem é mais comum quando você está lidando com um valor que não é inicialmente uma string ou quando você deseja garantir que qualquer tipo (não apenas um Double) possa ser convertido para String e, em seguida, para Double.

1. xxx.toString().toDouble()

Exemplo:

```
val number = 42
val doubleValue = number.toString().toDouble()
// Resultado: 42.0
```

1. xxx.toString().toDouble()

- **Quando usar:** Use este método quando você precisa garantir que o valor seja primeiro convertido para uma string e depois para um número Double. É útil em casos em que você está trabalhando com tipos que não são diretamente Double e precisa garantir que eles sejam convertidos para String antes da conversão.

2. xxx!!.toDouble()

- **Descrição**: O operador !! é usado para garantir que a variável xxx não é null. Se xxx for null, uma exceção NullPointerException será lançada. Após garantir que xxx não é null, você chama toDouble() diretamente.
- **Uso típico:** Este método é mais direto e é usado quando xxx já é de um tipo que possui o método toDouble() (como Int, Float, String que representa um número, etc.). O !! não faz parte da conversão, mas é usado para garantir que a variável não é null.

2. xxx!!.toDouble()

Exemplo:

val number: Int? = 42

val doubleValue = number!!.toDouble() //
Resultado: 42.0

Fatec
Diadema
Luigi Papaiz

2. xxx!!.toDouble()

- **Quando usar:** Use este método quando você tem certeza de que a variável não é null e que é de um tipo que já pode ser convertido diretamente para Double com o método toDouble(). Certifique-se de que o valor não é null antes de usar o !! para evitar exceções.

Resumo

- 1. xxx.toString().toDouble(): Converte o valor para String e depois para Double. Pode ser usado com qualquer tipo que suporte toString(), mas é menos direto e pode ser desnecessário se você já tem um valor numérico.

Resumo

- 2. xxx!!.toDouble(): Converte diretamente para Double, assumindo que xxx não é null e que o tipo de xxx suporta toDouble(). É mais direto e eficiente quando você já está lidando com tipos numéricos.
- Escolha a abordagem que se adapta melhor ao seu cenário específico, considerando se você precisa ou não lidar com null e o tipo de dado com o qual está trabalhando.

Operador in em Double

- No Kotlin, o operador in é comumente usado para verificar se um valor está dentro de um intervalo (range) ou de uma coleção.
- No entanto, a aplicação do operador in para Double não é tão direta quanto para tipos inteiros ou coleções, mas ainda é possível.

Guia Essencial: Funções na Linguagem Kotlin

Este guia resume os conceitos fundamentais da **Aula 05 de Programação para Dispositivos Móveis I.** Ele destaca como o Kotlin simplifica a declaração de funções, oferece flexibilidade com argumentos e permite o uso de funções como cidadãs de primeira classe.

Estrutura e Flexibilidade

Funções Top Level

Diferente do Java, o Kotlin permite criar funções fora de classes, simplificando a estrutura do código.

Retorno Unit Opcional

O tipo Unit indica que a função não retorna valor e sua declaração é opcional.

Parâmetros Padrão e Nomeados

Defina valores iniciais para argumentos e chame funções fora de ordem usando os nomes dos parâmetros.

Value

Name

Recursos Avançados e Conversão

Argumentos Variáveis (vararg)

Use o prefixo vararg para permitir que uma função aceite um numero indefinido de argumentos.

Funções como Parâmetro

Utilize o operador `::` para passar uma função como argumento para outra (Higher-Order Functions).

Conversão para Double

Conversão Direta

Object → String → Double

Use `.toString().toDouble()` para tipos genéricos. Converte para String e depois para Double. Tipos que não são inicialmente numéricos.

Garantia de não-nulo

Int → !! → Double

Use `!!.toDouble()` para garantir que valores não-nulos sejam convertidos. Garante que não é nulo e converte diretamente. Variáveis nullable de tipos numéricos (Int, Float).

O que caracteriza uma função como sendo de 'Top Level' na linguagem Kotlin?

A. A função só pode ser acessada por outras classes do mesmo pacote.

B. A função é declarada fora de qualquer classe.

C. A função deve ser declarada dentro de uma classe estática.

D. A função é obrigatoriamente a primeira a ser executada no programa.

B. A função é declarada fora de qualquer classe.

✓ **Isso mesmo!**

Funções de Top Level são definidas diretamente no arquivo, sem a necessidade de estarem encapsuladas em uma estrutura de classe.

Sobre o tipo de retorno $Unit$ em Kotlin, qual das seguintes afirmações é verdadeira?

A. O tipo $Unit$ é utilizado para funções que retornam o valor zero.

B. É obrigatório declarar explicitamente o retorno $Unit$ em todas as funções sem retorno.

C. Ele indica que a função não retorna um valor significativo.

D. Uma função $Unit$ pode retornar qualquer tipo de objeto, desde que seja nulo.

C. Ele indica que a função não retorna um valor significativo.

✓ **Isso mesmo!**

O $Unit$ no Kotlin é o equivalente ao 'void' de outras linguagens, representando a ausência de um valor de retorno.

Qual é a principal vantagem da utilização de 'Argumentos Nomeados' ao chamar uma função?

A. Permitir que a função aceite uma quantidade ilimitada de parâmetros.

B. Permitir a passagem de argumentos em uma ordem diferente daquela definida na assinatura da função.

C. Garantir que os parâmetros sejam convertidos automaticamente para o tipo String.

D. Aumentar a velocidade de execução do código compilado.

B. Permitir a passagem de argumentos em uma ordem diferente daquela definida na assinatura da função.

✓ **Isso mesmo!**

Ao especificar o nome do parâmetro, o desenvolvedor ganha flexibilidade para organizar os argumentos conforme a necessidade.

Considere a função: $funpotencia(base : Int = 2, expoente : Int = 1) : Int$. O que acontece se chamarmos $potencia(10)$?

A. O valor 10 é atribuído à 'base' e o 'expoente' assume o valor padrão 1.

B. O valor 10 é atribuído ao 'expoente' e a 'base' assume o valor padrão 2.

C. A função retorna um erro de execução (Runtime Error).

D. O código gera um erro de compilação por falta de argumentos.

A. O valor 10 é atribuído à 'base' e o 'expoente' assume o valor padrão 1.

✓ **Isso mesmo!**

Na ausência de nomes, os argumentos são atribuídos sequencialmente, preenchendo os parâmetros restantes com seus valores $default$.

Para que serve a palavra-chave $vararg$ na declaração de parâmetros de uma função em Kotlin?

A. Declarar que a variável pode ter seu valor alterado (mutável) dentro da função.

B. Indicar que o parâmetro é opcional e pode ser nulo.

C. Permitir que a função receba um número variável de argumentos de um mesmo tipo.

D. Forçar a conversão de todos os argumentos para o tipo Double.

C. Permitir que a função receba um número variável de argumentos de um mesmo tipo.

✓ **Isso mesmo!**

O $vararg$ transforma os múltiplos argumentos passados em um array dentro do corpo da função.

Ao passar uma função como parâmetro para outra, o que representa a notação ::?

A. Uma instrução para executar a função imediatamente antes de passá-la.

B. Um marcador que indica que a função possui múltiplos retornos.

C. Um operador de comparação lógica para verificar se duas funções são iguais.

D. Uma referência de função para que ela possa ser passada como um objeto.

D. Uma referência de função para que ela possa ser passada como um objeto.

✓ **Isso mesmo!**

O operador de referência de membro (::) permite que funções nomeadas sejam tratadas como parâmetros de alta ordem.

Qual é o efeito do uso do operador !! antes de chamar o método $.toDouble()$ em uma variável?

A. Ele afirma que a variável não é nula, lançando uma exceção caso ela seja null.

B. Ele converte automaticamente um valor nulo para 0.0.

C. Ele arredonda o valor numérico para a casa decimal mais próxima.

D. Ele verifica se a variável é uma String antes de tentar a conversão.

A. Ele afirma que a variável não é nula, lançando uma exceção caso ela seja null.

✓ **Isso mesmo!**

O operador de asserção não-nula (!!) ignora a segurança de tipos do Kotlin para garantir que o valor existe.

Por que utilizar $xxx.toString().toDouble()$ pode ser preferível em relação a métodos de conversão direta em certos contextos?

A. Porque é a forma mais rápida e eficiente de processamento no Kotlin.

B. Para garantir que tipos que não possuem o método $.toDouble()$ nativo possam ser convertidos via representação textual.

C. Porque o método $.toString()$ remove automaticamente caracteres não numéricos.

D. Para evitar que o programa lance exceções em caso de valores inválidos.

B. Para garantir que tipos que não possuem o método $.toDouble()$ nativo possam ser convertidos via representação textual.

✓ **Isso mesmo!**

Muitas classes possuem representação em string, permitindo uma ponte de conversão para tipos numéricos quando o método direto não existe.

Na linguagem Kotlin, como é definida uma função que não possui corpo e retorna o resultado de uma expressão diretamente?

A. Utilizando a palavra-chave 'expression' antes de 'fun'.

B. Declarando a função dentro de um bloco 'static'.

C. Omitindo o nome da função e os parâmetros.

D. Utilizando o sinal de igualdade (=) após a assinatura da função.

D. Utilizando o sinal de igualdade (=) após a assinatura da função.

✓ **Resposta correta**

Funções de expressão única eliminam a necessidade de chaves e da palavra-chave 'return', tornando o código mais conciso.

Qual é a principal diferença entre Kotlin e Java mencionada no material em relação ao escopo das funções?

A. As funções no Kotlin são sempre públicas por padrão, enquanto no Java são privadas.

B. No Java não existem funções de Top Level, enquanto no Kotlin elas são permitidas.

C. O Kotlin não permite funções dentro de classes, ao contrário do Java.

D. No Java as funções podem retornar múltiplos valores, o que é proibido no Kotlin.

B. No Java não existem funções de Top Level, enquanto no Kotlin elas são permitidas.

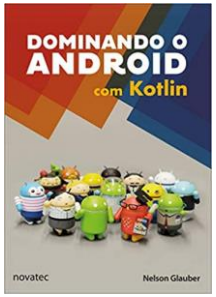
✓ **Isso mesmo!**

O Java exige que toda função (método) pertença obrigatoriamente a uma classe ou interface.

- Atividade 03

Referências

- GLAUBER, Nelson. Dominando o Android com Kotlin, 3ª Ed. São Paulo: Editora Novatec, 2019
- LECHETA, Ricardo R. **Android Essencial com Kotlin**. 2ª ed. São Paulo: Editora Novatec, 2018



Obrigado!

Prof. Marcos Vasconcelos de Oliveira

Fatec
Diadema
Luigi Papaiz